

Mining Big Data Streams

Module 4

Stream-Management System

- A **Stream-Management System (SMS)** is designed to handle and process continuous streams of data in real-time. Unlike traditional databases that store static data and process queries on a fixed dataset, a stream-management system deals with dynamic data that continuously flows in. These systems are crucial for applications that require real-time data processing, such as sensor data analysis, social media feeds, financial transactions, and network monitoring.
- Key features of a Stream-Management System:
- **Real-time Processing:** Data is processed as it arrives.
- **Continuous Queries:** Queries run continuously over the streaming data rather than running once on a static dataset.
- **Windowed Operations:** Stream processing often involves looking at a subset of data (e.g., last 5 minutes) rather than processing the entire stream.

Examples of Stream Sources

1. Sensor Networks:

1. Sensors continuously generate data about temperature, humidity, speed, pressure, etc., in real-time.
2. **Example:** IoT (Internet of Things) devices monitoring weather conditions or smart city sensors tracking traffic and pollution.

2. Social Media:

1. Social platforms like Twitter, Facebook, and Instagram generate a constant stream of data such as posts, likes, comments, and shares.
2. **Example:** Twitter's data feed can be used to monitor trending topics or real-time events.

3. Financial Transactions:

1. Real-time data is produced during stock trades, bank transactions, credit card usage, and cryptocurrency exchanges.
2. **Example:** A stock market feed continuously streams stock prices and trade information.

4. Network Monitoring:

1. Continuous data streams come from monitoring computer networks, which generate logs of system events, security alerts, and traffic data.
2. **Example:** Monitoring data packets in a network to detect anomalies such as cyber-attacks.

5. Clickstream Data:

1. Data streams generated from user clicks on websites, showing user behavior and interactions.
2. **Example:** E-commerce websites tracking what users click on, what they view, and what they add to the cart.

What is DSMS Architecture?

- DSMS stands for **data stream management system**. It is nothing but a software application just like [DBMS](#) (database management system) but it involves processing and management of a continuously flowing data stream rather than static data like Excel PDF or other files. It is generally used to deal data streams from with various sources which include sensor data, social media fields, financial reports, etc.
- Just like DBMS, DSMS also provides a wide range of operations like storage, processing, analyzing, integration also helps to generate the visualization and report only used for data streams.
- There are wide range of DSMS applications available in the market among them Apache Flint, [Apache Kafka](#), [Apache Storm](#), Amazon kinesis, etc. DSMS processes 2 types of queries standard queries and ad hoc queries.

Stream Data Processing System

- Each stream is handled independently but can be processed together.

StreamProcessor:

- This is the **core processing unit**.
- It performs real-time analysis using:
 - **Standing queries**: Predefined queries that constantly run on incoming data.
 - **Ad hoc queries** (connected from point 5): User-defined custom queries submitted on-demand.
- It decides whether data should be passed as output or stored.

Storage Systems:

Two types:

Limited Working Storage: Temporarily stores data for short-term processing.
Example: Sliding window operations (e.g., last 10 seconds of data).

Archival Storage: Saves historical data for long-term use or future analysis.
Example: Backup, analytics, compliance.

Ad Hoc Queries:

These are **custom, user-triggered queries** (not pre-written).

Example: A user wants to know the number of error logs in the last hour – this query is submitted to the stream processor.

Output Stream:

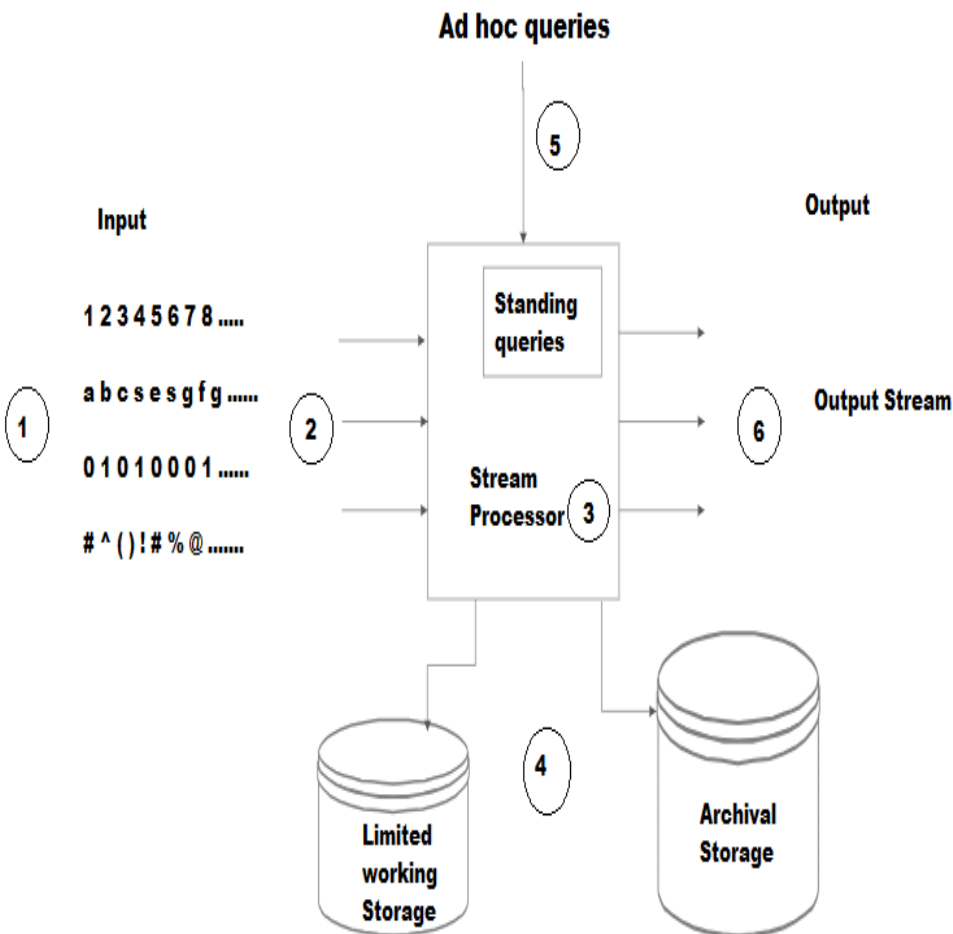
Final processed data is pushed out.

Can go to:

Dashboards (real-time display),

Alert systems (e.g., if temperature is too high),

Other systems for further action.



Real-Life Example Mapping (like Uber):

Diagram Component	Uber Example
Input streams	Cab GPS data, speed, customer feedback
Stream processor	Calculates ETA, detects fraud
Working storage	Stores data from last few minutes
Archival storage	Keeps full trip histories
Ad hoc query	Operator checks how many cabs are idle in Mumbai
Output stream	Updated alerts, map updates

Key Differences Between Batch and Stream Processing

Aspect	Batch Processing	Stream Processing
Data Collection	Data is collected in batches (chunks) over time.	Data is processed continuously as it arrives.
Latency	High latency (processing occurs after data collection).	Low latency (processing happens in real time).
Processing Time	Processes large chunks of data at once.	Processes data event by event or record by record.
Use Cases	Suitable for non-time-sensitive tasks like payroll or reporting.	Suitable for real-time applications like fraud detection or live monitoring.
Complexity	Typically simpler but requires scheduling and batch intervals.	More complex to implement, but provides real-time insights.
Fault Tolerance	Fault tolerance is built into most batch systems due to scheduled intervals.	Stream processing requires more sophisticated fault-tolerance mechanisms to ensure no data is lost.
Storage Requirements	Data is usually stored until the batch is processed.	Data is processed immediately and often discarded afterward (or stored only if needed).
Examples	Payroll system, billing at month end, offline data analysis.	Fraud detection, live traffic monitoring, stock price updates.

Stream Queries

- Stream Queries are **continuous queries** that work on **live incoming data streams**.

They **do not wait** for all data to be collected, unlike traditional (batch) queries. Instead, they run **as data arrives** — useful for **real-time analytics** or triggering **instant actions**..

Key Features of Stream Queries:

1. Continuous Execution:

1. The query keeps running nonstop.
2. It keeps updating the results automatically as new data comes in.

2. Windowing:

1. Because data streams are **infinite**, queries often use *windows* to limit the data they process at a time.

3. Two common window types:

1. Sliding Window:

1. Operates on a **rolling subset** of data.
2. Example: “Last 10 minutes of temperature data” – as new data comes in, old data slides out.

2. Tumbling Window:

1. Operates on **fixed-size chunks** of time with **no overlap**.
2. Example: Every 5 minutes, a new batch of data is processed separately.

Real-Life Examples:

- **Real-Time Monitoring:**

A stream query can **constantly calculate the average temperature** from a sensor feed over the last 10 seconds.

- **Event Detection:**

A stream query can monitor social media data and **detect sudden spikes in hashtag usage**, helping identify **trending topics instantly**.

Data Stream Operations in the Context of Big Data

- When working with data streams, the challenge is to efficiently process and analyze the data as it arrives. Traditional batch processing techniques don't work because the data is never at rest, and processing needs to be real-time. Hence, specialized operations are used for stream processing.

Common Data Stream Operations:

1. Filtering:

Filtering out unnecessary data from the stream, keeping only relevant or useful records.

Example: In a stream of sensor data, we might filter out readings that fall outside a specified threshold (e.g., temperatures below 20°C).

2. Aggregation:

Collecting and summarizing the data over a window of time (e.g., calculating averages, sums, counts, or maximum values).

Example: In stock market data, you might aggregate stock prices to calculate the average price over the last 5 minutes.

Windowing:

Breaking the continuous data stream into finite chunks or "windows" to perform operations on a subset of the stream.

•Types:

- **Sliding Window:** Overlapping time intervals that allow analyzing recent data repeatedly.
- **Tumbling Window:** Non-overlapping time intervals.

Example: In social media analytics, you may analyze user activity in 10-minute sliding windows to detect real-time trends.

Joining:

- Combining two or more streams of data, often using some form of a key to link the data records.
- **Example:** Joining a stream of user transactions with a stream of product data to enrich the transaction data with product details.

Counting:

- Counting the occurrences of specific events in the stream, such as clicks, likes, or errors.
- **Example:** Counting the number of failed transactions within a specific time window in an e-commerce platform.

Map/Transform:

- Applying a function to each element in the stream to transform or map it to a new format or value.
- **Example:** Converting temperature readings from Celsius to Fahrenheit as they stream from IoT sensors.

Reducing:

Aggregating the entire stream or chunks of it into a single value.

- **Example:** Reducing a stream of sales data to compute the total revenue within a given window.

- Sampling:**

- Extracting a subset of elements from the stream, useful when processing all elements is too costly or impractical.

- Example:** In a web server log, sampling 10% of the incoming traffic to analyze user behavior without processing all the logs.

- Merging:**

- Combining multiple streams into a single unified stream.

- Example:** Merging streams from different IoT sensors to get a comprehensive view of all sensor data in one stream.

- Pattern Recognition:**

- Detecting specific patterns or sequences in the data stream.

- Example:** Detecting fraudulent transactions by identifying patterns of abnormal behavior in a stream of credit card transactions.

Real-World Application (like Amazon):

Operation

Use Case Example

Filter

Remove failed payment attempts

Map

Convert timestamps into readable date format

Window

Count number of items sold per 10 seconds

Join

Match live product view with inventory info

Aggregate

Sum total sales by category

Group By

Group user activity by device type

Sort

Show trending products by order count

Issues in Stream Processing

- Processing data streams in real-time poses several challenges that need to be addressed for effective system design.

1. Latency:

1. One of the key challenges in stream processing is reducing latency, i.e., the delay between data arriving and being processed. Systems must ensure that data is processed as close to real-time as possible.
2. **Example:** In financial markets, even a millisecond of delay in processing stock price changes could lead to significant financial losses.

2. Fault Tolerance:

1. Stream processing systems must be able to recover from failures without losing data or processing state. Continuous data flow means the system must handle network failures, node crashes, or software errors without compromising the stream's integrity.
2. **Example:** If a node in a cluster goes down while processing real-time sensor data, the system should reroute the data to a healthy node.

3. Data Ordering and Synchronization:

1. Data streams from multiple sources might arrive out of order or with delays. Ensuring that data is processed in the correct sequence can be difficult.
2. **Example:** In an online advertisement system, if clickstream data arrives out of order, it may affect the accuracy of user behavior analysis.

4. Scalability:

1. Stream processing systems must be able to handle increasing amounts of data and scale horizontally (add more machines) to maintain performance.
2. **Example:** A system tracking global social media posts on Twitter may need to scale as the volume of posts during a major event increases dramatically.

Stream Processing Frameworks in Big Data:

- Given the need for real-time processing, several frameworks have been developed to handle data streams in Big Data systems:

1. Apache Kafka:

- Primarily used for building real-time streaming data pipelines and applications.
- It provides a publish-subscribe messaging system where data streams are continuously processed.

2. Apache Flink:

- Provides true real-time, low-latency stream processing.
- Flink supports event-driven applications with stateful processing and windowing.

3. Apache Spark Streaming:

- Extends Apache Spark's batch processing engine to handle real-time streaming data.
- It works by dividing the continuous stream into small batches for processing.

4. Apache Storm:

A distributed real-time computation system that processes unbounded streams of data with low latency.

Decaying Windows in Stream Processing

- In stream processing, a **decaying window** (or **decaying weight** model) is used to assign progressively less importance to older data while keeping more recent data highly relevant.
- This technique allows real-time systems to prioritize newer events, helping to make more relevant decisions and predictions based on the most current data, while still considering historical trends to some extent.
- **Key Characteristics:**
 - **Gradual Decay:** Data points receive a "weight" that decreases over time, reducing their influence as they get older.
 - **Recent Data Focus:** More recent data points are given higher priority in processing and analysis.
 - **Avoids Hard Cutoffs:** Unlike fixed or sliding windows that discard data after a specific time, decaying windows maintain all data but reduce its importance gradually.
- **Scenario: Real-time traffic monitoring**
 - Vehicles send speed data every few seconds.
 - We want to monitor **traffic congestion**.
 - A **decaying window** can:
 - Give higher weight to the **last 2 minutes** of data.
 - Give lower weight to data from **3–5 minutes ago**.
 - This way, decisions reflect the **current road condition**, not old info.

The exponential decay formula is commonly used:

$$\text{weight} = e^{-\lambda \cdot t}$$

- t = time since the event
- λ = decay rate
- More recent events (low t) get higher weight

How It's Used in Practice

◆ 1. Recommendation Systems

- Give more weight to what the user **clicked recently**
- Decay older interests automatically over time

◆ 2. Stock Market Trend Analysis

- Latest price movements are weighted more than older ones
- Helps in short-term predictions while retaining some historical context

◆ 3. Network Monitoring

- Recent packet loss has higher priority than older stable periods
- Better for dynamic alerting and congestion control

Decaying Windows in Fraud Detection:

- In **credit card fraud detection**, a bank might monitor a stream of transactions in real time. The most recent transactions are much more critical in determining potential fraud than older ones.
- **Recent Transactions:** If the cardholder just made three unusual large purchases in the last minute, the system will treat these transactions with more weight.
- **Older Transactions:** Transactions from 12 hours ago will still be considered but will have much lower influence on the decision.

Benefit: This allows fraud detection systems to flag abnormal behavior based on recent activities while not completely discarding past behaviors that may also indicate trends. Decaying windows ensure that the system balances short-term spikes in activity with the cardholder's historical usage patterns.

Decaying Windows Example: Website Activity Monitoring

- Imagine you are running an e-commerce website and want to monitor **user activity** to provide real-time recommendations or detect anomalies (such as fraudulent transactions or traffic spikes).
- You receive a continuous stream of events such as **page views**, **clicks**, and **purchases** from users.
- You need to predict traffic patterns or detect abnormal behaviors, giving more importance to recent user activity while still keeping some historical data in mind.

Practical Example:

- If a user purchases 10 items in the last 5 minutes, and a different user purchased 10 items in the last 5 hours, you may want to treat the first user as more relevant for real-time personalization (such as offering more targeted recommendations).
- With decaying windows, the **5-minute activity** will have a higher weight (due to the recency), while the **5-hour activity** will have a much smaller weight, reducing its impact on immediate decisions.

Sampling Data in a Stream: Sampling Techniques

- In **stream processing**, where data continuously flows in real-time, it is often impractical to store and process the entire dataset due to resource limitations such as memory and processing power.
- **Sampling** allows systems to reduce the data volume while still capturing important trends and patterns. Sampling techniques aim to extract representative subsets of data from the entire stream, enabling analysis that is computationally efficient but still meaningful.
- **Why Sampling is Necessary in Data Streams**
- **High Data Volume:** Data streams can generate massive amounts of data, making it infeasible to process every single data point.
- **Resource Constraints:** Limited memory and CPU resources mean that processing and storing all data points is often impossible.
- **Real-time Requirements:** Streaming applications typically need quick responses. Sampling helps by reducing the amount of data that needs to be processed in real-time.
- **Approximation of Trends:** Sampling provides a way to approximate trends and statistical properties of the entire dataset without processing the full stream.

Common Sampling Techniques

1. Reservoir Sampling:

- This technique is used to sample a fixed-size subset from a data stream where the total number of elements is unknown or very large.
- **How It Works:** As elements of the stream arrive, each element has a decreasing probability of being included in the sample.

Initially, the first k elements (where k is the sample size) are added to the sample. For the next incoming element (n -th element), it is included with probability k/n . If it is selected, it replaces a randomly chosen element from the current sample.

- **Advantages:** It ensures that every element of the stream has an equal chance of being selected, irrespective of the stream's length.
- **Example:** Consider a stream of 1 million records where we need a sample of 1000 records. Reservoir sampling would dynamically select which records to keep as more records arrive in the stream.

- Stream: [10, 60, 30, 40, 50, 60, 70, 80]
- Sample size $k = 3$ (we want to randomly select 3 numbers)

We'll go item by item using **Reservoir Sampling** logic.

Step 1: Add first 3 elements directly to the reservoir

Reservoir = [10, 60, 30]

Step 2: Process remaining elements (index 3 to 7)

Index 3 $\rightarrow$ Value = 40

Total items seen = 4

Generate random number between 1 and 4 $\rightarrow$ say 2 Since $2 \leq 3$,

replace item at index 2 in reservoir

Reservoir = [10, 40, 30]

Step by step process of Reservoir Sampling

Sample 3 elements from a data stream of 10 elements [1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

J – indicates the position (index) where to replace the new element.

Stream Element	i	Generated j	Action	Reservoir
1	1	-	Initialize	[1, -, -]
2	2	-	Initialize	[1, 2, -]
3	3	-	Initialize	[1, 2, 3]
4	4	2	Replace 2nd with 4	[1, 4, 3]
5	5	5	Do not replace	[1, 4, 3]
6	6	1	Replace 1st with 6	[6, 4, 3]
7	7	3	Replace 3rd with 7	[6, 4, 7]
8	8	2	Replace 2nd with 8	[6, 8, 7]
9	9	1	Replace 1st with 9	[9, 8, 7]
10	10	3	Replace 3rd with 10	[9, 8, 10]

→ J should be less than 3 , otherwise no replacement will take place. As the size of reservoir is 3

2. Bernoulli Sampling:

- Each element in the stream is sampled independently with a fixed probability p . This results in a sample where approximately $p\%$ of the data is kept.
- **How It Works:** For each incoming element in the stream, a random number between 0 and 1 is generated. If this number is less than the threshold p , the element is included in the sample. Otherwise, it is discarded.
- **Advantages:** Simple and easy to implement. The size of the sample varies depending on the stream, but it maintains a constant sampling probability.
- **Example:** In a financial transaction monitoring system, every transaction could be sampled with a 10% chance ($p = 0.1$) for auditing purposes

Example: Financial Auditing System

- Imagine a **bank processes 1 million transactions/day**.
- The audit team wants to **randomly review ~10% of them**.
- Set **$p = 0.1$** .
- For each transaction:
 - Generate a random number.
 - If the number is < 0.1 , send it for manual review.
- This way:
- Auditors get a **representative** sample.
- It ensures **variety and randomness** in samples.
- No need to store all transactions—just audit-worthy ones.

- Stratified Sampling:**

- The stream is divided into different "strata" or groups based on some attribute, and then sampling is done within each group independently.
- How It Works:** Before sampling, the data is partitioned into several substreams (or strata) based on a predefined attribute (e.g., geographic location, transaction type). A fixed number of samples or a percentage is taken from each group, ensuring that all groups are represented in the final sample.
- Advantages:** Ensures that each subgroup of the data is adequately represented in the sample, which can be important when certain groups are more important or smaller in size.
- Example:** In a survey, users can be grouped by age (18-25, 26-35, etc.), and a sample is taken from each group to ensure equal representation across all age brackets.

- Sliding Window Sampling:**

- In this technique, a fixed-size "window" of the most recent data is always kept for processing. As new data enters, old data exits the window.
- How It Works:** A window of size W is maintained. Only the last W elements of the stream are processed or sampled at any given time. As each new element arrives, the oldest element in the window is discarded.
- Advantages:** Suitable for real-time monitoring of the most recent data. Allows for analyzing only the latest trends.
- Example:** In a live temperature monitoring system, only the last 5 minutes of temperature data are kept and analyzed, while older data is discarded.

Bloom Filter

- A **Bloom Filter** is a probabilistic data structure used to test whether an element is a member of a set. It allows for space-efficient storage and can quickly tell whether an element is **possibly in the set** or **definitely not in the set**. However, it does not store the actual elements.

Key Characteristics:

- **Space Efficiency:** Uses much less space than traditional data structures.
- **False Positives:** Can return false positives (indicating that an element is in the set when it isn't) but never false negatives (indicating that an element is not in the set when it is).
- **No Deletion:** Once an element is added, it cannot be removed from a Bloom filter without affecting other elements.

How It Works

1. Hash Functions:

1. A Bloom filter uses multiple hash functions. Each function hashes an input and generates an index for a bit array.

2. Bit Array:

1. A fixed-size array of bits initialized to 0. The size of the bit array is chosen based on the expected number of elements and the desired false positive rate.

3. Adding Elements:

1. To add an element, the element is passed through each of the hash functions to obtain multiple indices. The bits at those indices in the bit array are set to 1.

4. Checking Membership:

1. To check if an element is in the set, the element is hashed with the same hash functions, and the corresponding bits are checked. If all bits are 1, the element is possibly in the set; if any bit is 0, the element is definitely not in the set.

Example of a Bloom Filter: Suppose we want to create a Bloom filter for the following elements: apple, banana, and orange.

1.Step 1: Initialize the Bloom Filter

- Let's say we create a Bloom filter with a bit array of size 10, all initialized to 0. **Bit Array:** [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

2.Step 2: Define Hash Functions

- Assume we have 3 hash functions:
 - $h_1(x) = (x \bmod 10)$ $h_1(x) = (x \bmod 10)$
 - $h_2(x) = (2x + 3) \bmod 10$ $h_2(x) = (2x + 3) \bmod 10$
 - $h_3(x) = (3x + 7) \bmod 10$ $h_3(x) = (3x + 7) \bmod 10$

3.Step 3: Add Elements

- **Adding apple** (assume the hash value for apple is 1, 3, 5):
 - Set bits at indices 1, 3, and 5.
 - **Bit Array:** [0, 1, 0, 1, 0, 1, 0, 0, 0, 0]
- **Adding banana** (assume hash values are 2, 4, 6):
 - Set bits at indices 2, 4, and 6.
 - **Bit Array:** [0, 1, 1, 1, 1, 1, 1, 0, 0, 0]
- **Adding orange** (assume hash values are 0, 1, 2):
 - Set bits at indices 0, 1, and 2.
 - **Bit Array:** [1, 1, 1, 1, 1, 1, 1, 0, 0, 0]

4.Step 4: Check Membership

- **Check if banana is in the filter:**
 - Hash values: 2, 4, 6. Bits at indices 2, 4, and 6 are all 1. **Result:** banana is possibly in the set.
- **Check if grape is in the filter:**
 - Hash values: 1, 3, 5. Bits at indices 1, 3, and 5 are all 1. **Result:** grape is possibly in the set (but it is actually not in the set).

Limitations of Bloom Filters

- **False Positive Rate:** The probability of false positives increases as more elements are added to the filter. The size of the bit array and the number of hash functions must be chosen carefully to minimize this.
- **No Deletion:** Once an element is added, it cannot be removed without risking the integrity of other elements.

Counting Bloom Filter

- A **Counting Bloom Filter** extends the basic Bloom filter by allowing elements to be added and removed from the filter. Instead of a bit array, it uses an array of counters. Each counter corresponds to a bit in the original Bloom filter, allowing for tracking the number of times an element has been added.

Key Characteristics:

- Allows for element removal, which is not possible in a traditional Bloom filter.
- Retains the probabilistic nature and can still produce false positives.

How It Works

1. Counting Array:

1. Instead of a bit array, a Counting Bloom filter uses an array of integers (counters). Each counter can be incremented or decremented.

2. Adding Elements:

1. When adding an element, the same hashing process is used as in the traditional Bloom filter, but instead of setting bits to 1, the corresponding counters are incremented.

3. Removing Elements:

1. To remove an element, the counters at the corresponding indices are decremented.

4. Checking Membership:

1. To check if an element is in the set, the counters are checked similarly to the bit array. If all counters are greater than 0, the element is possibly in the set; if any counter is 0, the element is definitely not in the set.

Example of a Counting Bloom Filter: Suppose we want to create a Counting Bloom filter for the same elements: apple, banana, and orange.

1.Step 1: Initialize the Counting Bloom Filter

- Let's say we create a Counting Bloom filter with a counter array of size 10, all initialized to 0. **Counter Array:** [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

2.Step 2: Define Hash Functions

- Assume we have the same 3 hash functions as before.

3.Step 3: Add Elements

- **Adding apple:** (hash values 1, 3, 5)
 - Increment counters at indices 1, 3, and 5.
 - **Counter Array:** [0, 1, 0, 1, 0, 1, 0, 0, 0, 0]
- **Adding banana:** (hash values 2, 4, 6)
 - Increment counters at indices 2, 4, and 6.
 - **Counter Array:** [0, 1, 1, 1, 1, 1, 1, 0, 0, 0]
- **Adding orange:** (hash values 0, 1, 2)
 - Increment counters at indices 0, 1, and 2.
 - **Counter Array:** [1, 2, 2, 1, 1, 1, 1, 0, 0, 0]

4.Step 4: Remove Elements

- **Removing apple:**
 - Decrement counters at indices 1, 3, and 5. **Counter Array:** [1, 1, 2, 0, 1, 0, 1, 0, 0, 0]
- **Removing banana:**
 - Decrement counters at indices 2, 4, and 6. **Counter Array:** [1, 1, 1, 0, 0, 0, 0, 0, 0, 0]

5.Step 5: Check Membership

- **Check if banana is in the filter:**
 - Hash values: 2, 4, 6.
 - Counters at indices 2, 4, and 6 are not all greater than 0.
 - Result: banana is definitely not in the set.
- **Check if apple is in the filter:**
 - Hash values: 1, 3, 5.
 - Counters at indices 1, 3, and 5 are not all greater than 0.
 - Result: apple is definitely not in the set.

Advantages of Counting Bloom Filter

- **Element Removal:** Supports addition and removal of elements without false positives.
- **Probabilistic Membership Testing:** Retains the space efficiency of Bloom filters while allowing for dynamic updates.

Limitations

- **Space Complexity:** Requires more space than a standard Bloom filter since it uses counters instead of bits.
- **Performance:** The need to manage counters can lead to higher computational overhead compared to a standard Bloom filter.

The Count-Distinct Problem

- The **Count-Distinct Problem** involves determining the number of unique elements in a large dataset or data stream.
- This problem is fundamental in various applications such as network traffic analysis, database optimization, web analytics, and more.

Why It Matters:

- **Scalability:** In big data environments, datasets can be enormous, making it impractical to store all elements to count unique items.
- **Real-Time Processing:** In streaming data scenarios, data arrives continuously and rapidly, requiring efficient algorithms to provide quick estimations.
- **Resource Efficiency:** Minimizing memory and computational resources is crucial when dealing with vast amounts of data.

Challenges:

- **High Volume:** The sheer number of elements makes exact counting infeasible.
- **Limited Memory:** Storing all unique elements to count them exactly would require prohibitive amounts of memory.
- **Speed:** The algorithm must process elements quickly to keep up with the data stream.

Applications:

- **Web Analytics:** Counting unique visitors to a website.
- **Database Management:** Estimating the number of distinct entries in large tables.
- **Network Security:** Identifying the number of unique IP addresses accessing a network.
- **Social Media:** Tracking unique hashtags or user interactions.

Flajolet-Martin Algorithm

The **Flajolet-Martin Algorithm** is a probabilistic algorithm designed to estimate the number of distinct elements in a data stream efficiently. It leverages hashing and bit manipulation techniques to provide a compact summary of the data stream, allowing for quick and memory-efficient estimations.

Key Characteristics:

- **Probabilistic Nature:** Provides an approximate count with a known error margin.
- **Space Efficiency:** Uses significantly less memory compared to exact counting methods.
- **Single Pass:** Processes the data stream in one pass, making it suitable for real-time applications.
- **Scalability:** Handles very large data streams effectively.

How It Works:

1. Hashing Elements:

- Each incoming element in the stream is passed through a hash function that maps it to a binary string (a sequence of bits). The hash function should uniformly distribute elements to minimize collisions.

2. Trailing Zeros:

- For each hashed binary string, identify the position of the first 1 bit (counting from the least significant bit). The number of trailing zeros before the first 1 is recorded.

3. Record Maximum Trailing Zeros:

- Keep track of the maximum number of trailing zeros observed across all hashed elements. This value (R) is used to estimate the number of distinct elements.

4. Estimation Formula:

- The number of distinct elements (E) is estimated using the formula: $E \approx 2^R$

**Incoming Element → Hash Function → Binary Representation → Count Trailing Zeros → Update Maximum R
→ Estimate E**

Practical Applications of FM algorithm:

1.Web Analytics:

1. Estimating the number of unique visitors to a website without storing all visitor IDs.

2.Database Systems:

1. Optimizing query performance by estimating distinct values in large tables.

3.Network Security:

1. Detecting the number of unique IP addresses accessing a network to identify potential threats.

4.Social Media Monitoring:

1. Counting unique hashtags or user interactions in real-time to track trends.

DGIM Algorithm

. The **DGIM Algorithm** (named after its creators Datar, Gionis, Indyk, and Motwani) is a probabilistic algorithm designed to efficiently estimate the number of 1s in the last N elements of a binary data stream. This is particularly useful for sliding window queries where exact counts are computationally expensive or infeasible due to high data velocity and volume.

Use Case:

- **Network Monitoring:** Counting active connections or failed login attempts within the last N seconds.
- **Database Systems:** Estimating recent query patterns.
- **Real-Time Analytics:** Monitoring real-time events, such as clicks or transactions, within a moving window.

Problem Statement: Counting 1s in a Sliding Window

Imagine you have a continuous binary data stream where each element is either a 0 or a 1. You need to maintain an estimate of the number of 1s in the last N elements (sliding window) at any given time.

Challenges:

- **High Throughput:** The data stream is too fast for exact counting.
- **Limited Memory:** Storing all N elements is impractical for large N .
- **Real-Time Processing:** Results need to be updated promptly as new data arrives.

Core Idea:

The DGIM algorithm maintains a compact summary of the stream by grouping consecutive 1s into buckets. Each bucket represents a group of 1s with the same size, and the algorithm ensures that for any bucket size, there are at most two buckets. This hierarchical grouping allows for an efficient approximation of the count within the sliding window.

Key Components:

- **Buckets:** Each bucket has a timestamp (indicating the most recent 1 in the bucket) and a size (number of 1s it represents).
- **Rules:**
 - **Bucket Size:** Powers of two (e.g., 1, 2, 4, 8, ...).
 - **Maximum Buckets per Size:** At most two buckets of the same size are allowed.
 - **Merging:** When adding a new bucket causes the maximum to be exceeded, the two oldest buckets of that size are merged into a new bucket of double the size.

RULES FOR FORMING THE BUCKETS:

1. The right side of the bucket should always start with 1. (if it starts with a 0, it is to be neglected) E.g. · 1001011 → a bucket of size 4, having four 1's and starting with 1 on its right end.
2. Every bucket should have at least one 1, else no bucket can be formed.
3. All buckets should be in powers of 2.
4. The buckets cannot decrease in size as we move to the left. (move in increasing order towards left)

<https://medium.com/fnplus/dgim-algorithm-169af6bb3b0c>

pls refer the above link for more details about DGIM alg

When the new bit = 1 enters:

New Bits

87		92		95		98		100		101	102		103	104
101011	000	10111	0	1	00	101	1	0	0	1			1	1
2^2		2^2		2^1		2^1	2^0			2^0				

(CREATING A NEW BUCKET OF
SIZE 1)

when next bit = 1 enters:

87	92	95	98	100	101	102	103	104
101011	000	10111	0	11	00	101	1	1
2^2	2^2	2^1	2^1	2^0	2^0	2^0	2^0	2^0

87	92	95	98	102	103	104		
101011	000	10111	0	11	00	101	1001	1
2^2	2^2	2^1	2^1	2^1	2^0			

87	92	98	102	103	104
101011	000	10111	1100101	1001	1
2^2	2^2	2^2	2^1	2^0	

Important concepts

- Decaying Windows
- different issues in data stream processing
- various data stream sources
- How do sensor networks and social media platforms contribute as sources of data streams?
- How does a stream query differ from a traditional SQL query?
- What is data stream. data stream operations in the context of big data
- main components of the DGIM algorithm?

- count-distinct problem in stream processing with suitable example?
- difference between batch processing and stream processing, highlighting the advantages of stream processing.
- scenarios where a Bloom filter might produce false positives and false negatives. How can these issues be mitigated?
- Explain what a Data Stream Management System (DSMS) is, and illustrate its architecture with a block diagram.
- stream queries are and describe the different categories of stream queries with examples.
- Imagine you're processing a binary stream to estimate the count of ones in a sliding window. Apply the concept of the DGIM algorithm to achieve this.
- reservoir sampling with other sampling techniques in stream processing.

- Given a scenario where you need to check the existence of an item in a large dataset, explain how you would use a Bloom filter to perform this check?
- Determine the distinct element in the stream using FM algorithm. Input stream of Integers= 1,3,2,1,2,3,4,3,1,2,3,1
 $h(x)=(6x+1)\bmod 5$
- Input stream of Integers= 4, 2, 5, 9, 1, 6, 3, 7 a) $h(x) = 3x + 1 \bmod 32$ b) $h(x) = x + 6 \bmod 32$
- Analyze the operation of a Bloom filter in detail. Provide a thorough explanation of how it works and illustrate its functionality with a practical example.
- Explain how the Flajolet-Martin algorithm can be used to estimate the number of distinct elements in a stream. Analyse the trade-offs between the Flajolet-Martin algorithm and the Datar-Gionis-Indyk-Motwani (DGIM) algorithm in terms of accuracy and memory space requirements.
- Compare and contrast the challenges of estimating the count of distinct elements in a stream with the challenges of counting ones in a sliding window. How do the algorithms address these challenges?
- Apply the concept of Bloom filters to a situation where you're processing incoming emails. Explain how a Bloom filter could be used to identify potential spam emails.
- A Bloom filter with $m=1000$ cells is used to store information about $n=100$ items, using $k=4$ hash functions. calculate the false positive probability of this instance"